



DER ZARATHUSTRA-BOT

EIN BOT FÜR ALLE UND KEINEN

Kurs: opencampus.sh / From LLMs to AI Agents

Projekt von Thorsten Köhler



Grundidee

- Spricht wie Friedrich Nietzsche in:

Also sprach Zarathustra. Ein Buch für Alle und Keinen, 1883–1885.

- Vermittelt dessen philosophische Lehre.

- Text online verfügbar:

<https://projekt-gutenberg.org/authors/friedrich-wilhelm-nietzsche/books/also-sprach-zarathustra-ein-buch-fuer-alle-und-keinen/>

Erstes Konzept: Text-Generation

- Developer Prompt definiert Sprechweise
- RAG wählt passende Textpassagen aus
- Antwort soll thematisch gut zur Nutzereingabe passen
- aber stets im Kontext der Vorlage bleiben
- Oder sollten nur Originalzitate verwendet werden?

Erster Versuch

- UI: Streamlit, Modell: OpenAI gpt-4.1-mini, Text: Zarathustras Vorrede.
- Erfolgreiches Imitieren des Sprachstils.
- Keine präzise Wiedergabe der Inhalte! Beispiel:

Bot sagt von sich selbst: „... um euch ein neues Licht zu bringen.“

Originaltext: Zarathustra spricht davon, dass *die Sonne* Licht bringt.

→ Aussagen werden etwas uminterpretiert.

→ Textverständnis der Nutzer wird verfälscht.

Problembehandlung

- Experimentieren mit Developer Prompt:

Sobald das LLM die Freiheit erhält, Text zu generieren, der im Original nicht wörtlich vorkommt, taucht das Problem wieder auf.

- Anpassung des Konzepts nötig:

LLM nicht mehr zum Imitieren des Sprachstils einsetzen!

Zweites Konzept: Text-Kombination

- Abwechslung durch Interaktion von LLM, Embedding Model, Zufall.
- Auf User-Eingabe folgt erste Retrieval-Anfrage an Embedding Model.
- Jedes Mal, wenn LLM einen Teil des Originaltexts erhält, wählt es daraus einen Satz aus.
- LLM entscheidet dann: Ausgabe oder Verlängerung.
- Falls Verlängerung: LLM übergibt letzten Satz an Zufalls-Tool, das Wörter zurückgibt; LLM startet damit neue Retrieval-Anfrage.
- LLM reiht alle ausgewählten Sätze aus dem Originaltext aneinander, bis es damit zufrieden ist oder Maximallänge erreicht; dann Ausgabe.
- Möglicherweise weitere Tools?

Lernerfahrungen (1)

- Nützlicher Git-Befehl: `git reset --soft origin/main`
(Wenn ich einen api key in der Datei hatte beim commit, ließ sich damit der lokale commit entfernen und Blockade aufheben.
Gut: Aktueller Arbeitsbaum/Stagingbereich bleibt erhalten.)
- Mein Eindruck: LLM kann sehr gut imitieren,
aber zum menschlichen „Denken“ gehört mehr.
- Maximale Anzahl von LLM-Zyklen begrenzen, damit keine unnötigen
Kosten entstehen durch sinnlos wiederholte Anfragen!

Lernerfahrungen (2)

- Developer Prompt in eigenem File speichern!
→ Übersichtlicher, leichter zu modifizieren, Versionen möglich.
- LLM benötigt sehr präzise, klare, direkte, explizite, kohärente und konsistente Anweisungen, insbesondere auch bei der Beschreibung der Tools!
- LLM im Developer Prompt anweisen, den Variablentyp bei Übergabe an Tool zu überprüfen, und wie es sich bei Abweichung verhalten soll!